Projekt 1 – świat klocków

Termin oddania: 21/21 października 2026.

Opis problemu

Na stole leżą klocki ponumerowane od 1 do N. Każdy klocek leży bezpośrednio na stole albo na innym klocku. Na jednym klocku może leżeć bezpośrednio co najwyżej jeden inny klocek, ale wieże mogą być dowolnie wysokie. Klocek jest **wolny**, jeśli nic na nim nie leży.

Przykładowy układ: 1 na stole, 2 na 1, 3 na stole, 4 na 2, 5 na 3, czyli dwie wieże (od dołu do góry): `[1 2 4]` i `[3 5]`.

Dozwolone ruchy (jeden ruch przenosi jeden klocek):

1. przełożenie wolnego klocka leżącego na innym klocku na stół;
2. przełożenie wolnego klocka leżącego na stole na **inny** wolny klocek;
3. *(tylko w wariancie rozszerzonym)* przełożenie wolnego klocka leżącego na innym klocku bezpośrednio na inny wolny klocek.

Projekt dotyczy dwóch wariantów zasad:

- **wariant podstawowy**: dozwolone ruchy 1–2 (zadania na 4 pkt),
- **wariant rozszerzony**: dozwolone ruchy 1–3 (zadania na 6 i 8 pkt).

Celem jest znalezienie sekwencji ruchów przeprowadzającej zadany układ początkowy w zadany układ docelowy. W zadaniach na 4 pkt sekwencja nie musi być najkrótsza; w zadaniach na 6 i 8 pkt algorytmy mają gwarantować najkrótszą sekwencję.

Formaty danych

Plik wejściowy zawiera dwie linie: układ początkowy i układ docelowy. Linie puste i zaczynające się od `#` są pomijane. Każdy układ to N liczb całkowitych oddzielonych spacjami; i-ta liczba to numer klocka, na którym leży klocek i, a `0` oznacza stół. Przykład:

```
# start: [1 2 4] [3 5]
0 1 0 2 3
# cel: wieża [5 4 3 2 1] (5 na dole)
2 3 4 5 0
```

Dla tego przykładu najkrótsza sekwencja ma **7 ruchów** w wariancie podstawowym i **5 ruchów** w wariancie rozszerzonym. Użyj go do sprawdzenia swojej implementacji.

Ruch zapisujemy jako parę `(klocek, cel)`, gdzie `cel = 0` oznacza stół, np. `(4, 0)` oznacza „połóż klocek 4 na stole”, a `(4, 5)` – „połóż klocek 4 na klocku 5”.

Zadania

Progi punktowe są kumulatywne: na 6 pkt trzeba wykonać także zadania na 4 pkt, a na 8 pkt – zadania na 4 i 6 pkt.

Zadania na 4 punkty (wariant podstawowy)

1. Napisz program, który wczytuje z pliku układ początkowy i docelowy w podanym formacie. Stan ułożenia klocków reprezentuj za pomocą wyodrębnionej klasy. Program ma odrzucać niepoprawne układy (klocek leżący na samym sobie lub na nieistniejącym klocku, dwa klocki na jednym klocku, cykl typu „1 na 2, 2 na 1") z czytelnym komunikatem.
2. Zaimplementuj algorytm DFS z nawrotami (backtrackingiem) znajdujący sekwencję ruchów przeprowadzającą jeden układ w drugi. Algorytm nie może wpadać w nieskończone pętle (zadbaj o pomijanie stanów już odwiedzonych). Wolno porządkować kolejność rozpatrywanych ruchów; opisz w raporcie, jak to zrobiono. Zastosuj limit liczby rozwiniętych węzłów lub czasu i zgłaszaj jego przekroczenie.
3. Napisz procedurę weryfikującą sekwencję ruchów względem układu początkowego. Procedura ma sprawdzić, czy każdy ruch jest dozwolony (przenoszony klocek jest wolny, docelowy klocek jest wolny, nie kładziemy klocka na nim samym, przestrzegamy zasad danego wariantu) oraz czy końcowy układ jest równy docelowemu. W przypadku błędu ma wskazać numer pierwszego niepoprawnego ruchu i przyczynę.
4. Wygeneruj losowe układy początkowe i docelowe dla każdego N od 3 do 15 (po 5 układów dla każdego N). Ustal i podaj ziarno generatora liczb losowych, aby wyniki dało się odtworzyć. W raporcie podaj dla każdego problemu: układy, długość znalezionej sekwencji, liczbę rozwiniętych węzłów, czas oraz wynik weryfikacji. Pełne sekwencje ruchów podaj dla $N \leq 8$.

Zadania na 6 punktów (wariant rozszerzony)

1. Rozszerz generowanie ruchów i procedurę weryfikacji o ruch 3. Wariant zasad ma być parametrem programu – oba warianty muszą nadal działać.
2. Zaimplementuj algorytm gwarantujący najkrótszą sekwencję ruchów, np. BFS.
3. Porównaj DFS z zadania 4.2 z algorytmem z punktu 2 na 10 losowych problemach z $N = 8$ w wariancie rozszerzonym. Dla każdego problemu podaj: długość znalezionej sekwencji, liczbę rozwiniętych węzłów, maksymalną liczbę stanów przechowywanych jednocześnie w pamięci oraz czas działania; podaj też średnie. Skomentuj wyniki.

Zadania na 8 punktów (wariant rozszerzony)

1. Zaimplementuj algorytm iterative deepening A* (IDA*) znajdujący najkrótszą sekwencję ruchów.

2. Jako pierwszą heurystykę zastosuj liczbę klocków, które leżą na innym podłożu niż w układzie docelowym (podłożem jest klocek albo stół). Uzasadnij, dlaczego ta heurystyka nigdy nie przeszacowuje kosztu.
3. Zaproponuj lepszą heurystykę, która nigdy nie przeszacowuje kosztu, i uzasadnij jej dopuszczalność. Porównaj obie heurystyki (liczba rozwiniętych węzłów, czas) na co najmniej 5 losowych problemach dla każdego $N \in \{6, 8, 10, 12, 15\}$, z limitem liczby rozwiniętych węzłów. Sprawdź, że dla tych samych problemów IDA* z obiema heurystykami zwraca sekwencje tej samej długości.

Pytanie dodatkowe (nieobowiązkowe): czy w wariancie podstawowym da się podać heurystykę dającą zawsze dokładną długość najkrótszej sekwencji? Co z tego wynika dla złożoności problemu w tym wariancie?

Wskazówki

Liczba wszystkich możliwych układów N klocków rośnie bardzo szybko:

N	3	5	8	10	12	15
liczba układów	13	501	394 353	$\approx 5,9 \cdot 10^7$	$\approx 1,2 \cdot 10^{10}$	$\approx 6,6 \cdot 10^{13}$

Wartości te wynikają ze wzoru

$$\sum_{k=1}^{N} \binom{N-1}{k-1} \frac{N!}{k!},$$

w którym k oznacza liczbę wież. Kolejność klocków w wieży ma znaczenie, a kolejność samych wież – nie.

Wyprowadzenie. Policzmy układy złożone z dokładnie k wież.

1. Ustawiamy wszystkie N klocków w jeden ciąg. Można to zrobić na $N!$ sposobów.
2. Rozcinamy ciąg na k niepustych, kolejnych fragmentów. Między sąsiednimi klockami jest $N-1$ miejsc, a cięcia wybieramy w $k-1$ z nich. Daje to $\binom{N-1}{k-1}$ możliwości. Każdy fragment jest jedną wieżą: klocki czytane od lewej leżą od dołu do góry.
3. W ten sposób każdy układ z k wieżami otrzymujemy dokładnie $k!$ razy, raz dla każdej kolejności wież w ciągu. Wieże stoją na stole bez ustalonej kolejności, więc dzielimy przez $k!$.

Układów z k wieżami jest więc $\binom{N-1}{k-1}\frac{N!}{k!}$. Są to tzw. liczby Laha. Sumując po $k = 1, \ldots, N$, otrzymujemy wzór powyżej.

Pełne przeszukanie przestrzeni stanów (DFS bez porządkowania ruchów, BFS) jest więc realne tylko dla małych N. Przekroczenie limitu przy większych N jest dopuszczalnym i wartym odnotowania wynikiem – nie trzeba go ukrywać.

Co oddać

- kod źródłowy z instrukcją uruchomienia;
- przykładowe pliki wejściowe;
- raport z wynikami i komentarzem do zadań z wybranego progu;
- projekt należy zaprezentować prowadzącemu w trakcie zajęć i odpowiedzieć na ewentualne pytania;
- *(zalecane)* testy automatyczne, np. sprawdzające, że każda znaleziona sekwencja przechodzi weryfikację i że algorytmy optymalne zwracają tę samą długość.